

Benchmarking de CPU e Memória

Objetivo

Executar benchmarks de CPU sob diferentes cargas, coletar métricas de desempenho e visualizar os resultados em gráficos. O aluno irá comparar o comportamento do sistema com 1, 2 e 4 núcleos de CPU ativos e analisar os resultados visualmente no Datawrapper.

Contexto: O que é benchmarking?

Benchmark é um teste padronizado que mede o desempenho de um sistema sob condições controladas. Na avaliação de desempenho, benchmarks são usados para:

- Comparar sistemas diferentes sob as mesmas condições
- Identificar gargalos de desempenho
- Verificar o impacto de mudanças de configuração
- Coletar dados para modelagem e análise

Neste laboratório, o benchmark vai simular carga de CPU e medir duas métricas fundamentais:

- Throughput — quantas operações o sistema consegue processar por segundo. Quanto maior, melhor.
- Latência — quanto tempo cada operação leva para ser concluída. Quanto menor, melhor.

Pré-requisitos

- Docker e Docker Compose instalados
- Conhecimento básico de terminal Linux

Como iniciar o laboratório

Abra um terminal nesta pasta e execute:

```
docker compose up -d
docker compose exec avd-lab-01 bash --login
```

Exercícios

Exercício 1 — Entendendo o benchmark

O sysbench é uma ferramenta profissional de benchmark usada para medir desempenho de CPU. O teste funciona assim: ele pede ao processador que calcule todos os números primos até um limite definido. Quanto mais rápido o processador, maior o número de eventos por segundo.

Execute um teste rápido para ver o formato da saída:

```
sysbench cpu --cpu-max-prime=20000 --threads=1 --time=5 run
```

Entendendo os parâmetros:

- `--cpu-max-prime=20000` — calcula primos até 20.000 (define a complexidade do teste)
- `--threads=1` — usa apenas 1 thread (1 núcleo de CPU)
- `--time=5` — roda por 5 segundos

Na saída, observe:

- `events per second` — throughput: quantas operações por segundo
- `avg latency` — latência média em milissegundos por operação
- `95th percentile` — 95% das operações terminaram em até esse tempo

Por que o percentil 95 importa? A média pode esconder picos de lentidão. Se a latência média é 0.20ms mas o percentil 95 é 0.50ms, significa que 5% das operações são muito mais lentas — o que em sistemas reais causa problemas visíveis ao usuário.

Exercício 2 — Executar o benchmark automatizado

O script `benchmark.sh` roda 3 cenários automaticamente e salva os resultados em CSV:

```
./benchmark.sh
```

Os 3 cenários testados são:

Cenário	Núcleos	Duração
Carga Leve	1	10s
Carga Média	2	10s
Carga Alta	4	10s

O que esperar? Com mais núcleos, o throughput tende a aumentar. Mas esse ganho raramente é linear — dobrar os núcleos não necessariamente dobra o desempenho, por causa de overhead de sincronização entre threads.

Exercício 3 — Verificar o arquivo CSV gerado

O script salva os resultados em resultados/benchmark.csv. Visualize o conteúdo:

```
cat resultados/benchmark.csv
```

O formato do CSV é:

```
cenario,nucleo,duracao_s,eventos_por_seg,latencia_media_ms,uso_cpu_percent
Carga Leve,1,10,...
Carga Media,2,10,...
Carga Alta,4,10,...
```

Copie toda a saída do comando — você vai colar no Datawrapper no Exercício 5.

Exercício 4 — Observar o sistema durante a carga

Abra um Terminal 2 nesta pasta e entre no container:

```
docker compose exec avd-lab-01 bash --login
```

No Terminal 2, inicie o monitor:

```
htop
```

No Terminal 1, rode o benchmark novamente:

```
./benchmark.sh
```

Observe no Terminal 2:

- As barras de CPU — quantas estão ativas e quanto de cada uma está sendo usado
- Como o uso de CPU muda entre os cenários de carga leve, média e alta
- A coluna %CPU dos processos em execução

Pressione q para sair do htop.

Exercício 5 — Visualizar os resultados no Datawrapper

O Datawrapper é uma ferramenta gratuita de visualização de dados, sem precisar de conta ou planilha.

Passo 1 — Acesse datawrapper.de e clique em Start creating.

Passo 2 — Na tela de upload, clique em Copy & paste data table e cole todo o conteúdo do arquivo CSV — com cabeçalho e todas as linhas:

```
cenario,nucleo,duracao_s,eventos_por_seg,latencia_media_ms,uso_cpu_percent
Carga Leve,1,10,...
Carga Media,2,10,...
Carga Alta,4,10,...
```

Passo 3 — Clique em Proceed. Na etapa Check & Describe, confirme que os dados aparecem corretamente na tabela e clique em Proceed novamente.

Passo 4 — Na etapa Visualize, escolha Column Chart (gráfico de colunas).

Passo 5 — Clique na aba Refine. Em Columns → Select column, escolha eventos_por_seg no dropdown. O gráfico vai mostrar o throughput de cada cenário.

Passo 6 — Para o gráfico de latência, troque o dropdown para latencia_media_ms e observe como o gráfico muda automaticamente.

Passo 7 — Para o gráfico de uso de CPU, troque para uso_cpu_percent.

O que observar nos gráficos:

- O throughput aumentou de forma linear com mais núcleos?
- A latência diminuiu, aumentou ou ficou estável?
- O uso de CPU acompanha proporcionalmente o número de núcleos?

Exercício 6 — Análise dos resultados

Com base nos gráficos gerados, responda:

1. Qual cenário obteve maior throughput? O resultado era esperado? Por quê?
2. O aumento de núcleos melhorou o desempenho de forma linear? 2 núcleos dobrou o throughput em relação a 1?
3. O que aconteceu com a latência ao aumentar os núcleos?
4. Se esse sistema fosse um servidor real, qual cenário de carga você escolheria para operação normal? Por quê?

Conceito: A relação entre carga e desempenho raramente é linear. Em teoria das filas — que você estudará nesta disciplina — isso é explicado pela Lei de Amdahl: o ganho de desempenho com paralelismo é limitado pela fração do trabalho que não pode ser paralelizada.

Como encerrar o laboratório

```
exit
docker compose down
```

Os resultados ficam salvos na pasta resultados/ mesmo após encerrar o container.